

02_session-py

#PostgreSQL #python

1 Ubicación del archivo dentro del proyecto

```
siem-lab/  
└─ backend/  
    └─ app/  
        └─ db/  
            └─ session.py
```

El archivo `session.py` se encuentra dentro del módulo de base de datos del backend:

```
backend/app/db/
```

Su función principal es proporcionar sesiones de base de datos a los endpoints de FastAPI mediante una dependencia llamada `get_db`.

Este archivo se apoya directamente en `database.py`, concretamente en la fábrica de sesiones:

```
SessionLocal
```

La relación entre ambos archivos es:

```
database.py  
  ↓  
define SessionLocal  
  ↓  
session.py  
  ↓  
define get_db()  
  ↓  
endpoints FastAPI
```

2 Comando utilizado para visualizar el archivo

```
cd ~/siem-lab  
sed -n '1,220p' backend/app/db/session.py
```

Desglose del comando:

```
cd ~/siem-lab
```

Sitúa la terminal en la raíz del proyecto.

```
sed
```

Ejecuta el programa `sed`, utilizado para leer o transformar texto.

```
-n
```

Evita que `sed` imprima todo el archivo automáticamente.

```
'1,220p'
```

Indica que se impriman las líneas de la 1 a la 220.

```
backend/app/db/session.py
```

Es la ruta del archivo que se quiere visualizar.

3 Código completo del archivo

```
from collections.abc import Generator
from .database import SessionLocal

def get_db() -> Generator:
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

4 Función general del archivo

El archivo `session.py` define la función `get_db`.

Esta función se utiliza como dependencia en FastAPI para proporcionar una sesión de base de datos a los endpoints.

En rutas como `health.py` o `metrics.py`, aparece este patrón:

```
db: Session = Depends(get_db)
```

Eso significa que FastAPI debe ejecutar `get_db()` para obtener una sesión SQLAlchemy y pasarla al endpoint.

La función `get_db()` realiza tres acciones principales:

1. Crea una sesión de base de datos usando `SessionLocal()`.
2. Entrega esa sesión al endpoint mediante `yield`.
3. Cierra la sesión al terminar la petición.

Este archivo evita repetir manualmente la apertura y cierre de sesiones en cada endpoint.

5 Estructura general del archivo

El archivo puede dividirse en tres bloques:

```
from collections.abc import Generator
```

Importación del tipo `Generator`.

```
from .database import SessionLocal
```

Importación de la fábrica de sesiones definida en `database.py`.

```
def get_db() -> Generator:
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

Definición de la dependencia que crea, entrega y cierra la sesión de base de datos.

Visualmente:

```
session.py
├─ Importación de Generator
├─ Importación de SessionLocal
└─ Función get_db()
```

- └─ Crea sesión
- └─ Entrega sesión
- └─ Cierra sesión

6 Análisis línea por línea

Importación de Generator

```
from collections.abc import Generator
```

Esta línea importa `Generator` desde `collections.abc`.

Desglose:

```
from collections.abc
```

Indica que se importa desde el módulo `collections.abc`, que contiene clases abstractas relacionadas con colecciones e iteradores.

```
import Generator
```

Importa el tipo `Generator`.

En este archivo se utiliza como anotación de tipo:

```
def get_db() -> Generator:
```

Un generador es una función que no devuelve un valor con `return`, sino que produce valores con `yield`.

En este caso, `get_db()` usa:

```
yield db
```

Por eso se anota como `Generator`.

Importación relativa de SessionLocal

```
from .database import SessionLocal
```

Esta línea importa `SessionLocal` desde el archivo `database.py`.

El punto inicial:

```
.
```

indica una importación relativa dentro del mismo paquete.

Como `session.py` y `database.py` están en la misma carpeta:

```
backend/app/db/
```

esta línea significa:

```
desde el archivo database.py de esta misma carpeta, importa SessionLocal
```

Equivalencia conceptual:

```
backend/app/db/session.py  
↓
```

```
backend/app/db/database.py
↓
SessionLocal
```

SessionLocal fue definido en database.py con:

```
SessionLocal = sessionmaker(bind=engine, autoflush=False, autocommit=False)
```

Es decir, SessionLocal es una fábrica capaz de crear sesiones SQLAlchemy conectadas a PostgreSQL.

Definición de la función get_db

```
def get_db() -> Generator:
```

Esta línea define la función get_db .

Desglose:

```
def
```

Palabra clave de Python para definir una función.

```
get_db
```

Nombre de la función.

El nombre indica su finalidad:

```
get_db → obtener base de datos / obtener sesión de base de datos
```

```
()
```

La función no recibe parámetros.

```
-> Generator
```

Anotación de tipo que indica que la función devuelve un generador.

Esto tiene sentido porque dentro de la función se utiliza:

```
yield db
```

```
:
```

Marca el inicio del bloque de código de la función.

Creación de la sesión

```
db = SessionLocal()
```

Esta línea crea una sesión de base de datos.

Desglose:

```
db
```

Variable local donde se guarda la sesión.

```
=
```

Operador de asignación.

```
SessionLocal()
```

Llama a la fábrica de sesiones importada desde `database.py`.

Aunque `SessionLocal` no es una sesión concreta, al ejecutarla con paréntesis:

```
SessionLocal()
```

se crea una nueva sesión SQLAlchemy.

La relación sería:

```
SessionLocal
  ↓
SessionLocal()
  ↓
db
```

A partir de este momento, `db` puede utilizarse para consultar o modificar la base de datos.

Ejemplos de uso en otros endpoints:

```
db.execute(...)
db.add(...)
db.commit()
db.refresh(...)
db.close()
```

Inicio del bloque `try`

```
try:
```

Esta línea inicia un bloque `try`.

El bloque `try` se usa para ejecutar código que debe ir acompañado de una limpieza posterior.

En este caso, la limpieza importante es cerrar la sesión de base de datos.

La estructura completa es:

```
try:
    yield db
finally:
    db.close()
```

Esto significa:

```
entrega la sesión al endpoint;
cuando termine, pase lo que pase, cierra la sesión.
```

Entrega de la sesión con `yield`

```
yield db
```

Esta línea es la parte central de `get_db`.

`yield` convierte la función en un generador.

A diferencia de `return`, que finaliza una función, `yield` entrega un valor y deja la función suspendida hasta que FastAPI termina de usar ese valor.

En este caso:

```
yield db
```

entrega la sesión de base de datos al endpoint que la ha solicitado.

Por ejemplo, en `health.py`:

```
def health(db: Session = Depends(get_db)):
```

FastAPI ejecuta `get_db()`, recibe el valor producido por `yield db` y lo asigna al parámetro `db`.

La relación es:

```
get_db()
  ↓
crea db = SessionLocal()
  ↓
yield db
  ↓
FastAPI entrega db al endpoint
  ↓
endpoint usa db
```

Bloque `finally`

```
finally:
```

Esta línea inicia un bloque `finally`.

En Python, el bloque `finally` se ejecuta siempre después del bloque `try`, tanto si todo ha ido bien como si se ha producido un error.

En este archivo, `finally` garantiza que la sesión se cierre incluso si el endpoint falla.

Esto es importante porque dejar sesiones abiertas puede provocar:

- Fugas de conexiones.
- Consumo innecesario de recursos.
- Bloqueos o saturación del pool de conexiones.

Cierre de la sesión

```
db.close()
```

Esta línea cierra la sesión de base de datos.

Desglose:

```
db
```

Variable que contiene la sesión SQLAlchemy.

```
.close()
```

Método que cierra la sesión.

Cerrar la sesión no significa necesariamente cerrar físicamente toda conexión con PostgreSQL para siempre. SQLAlchemy puede devolver la conexión al pool para reutilizarla.

Lo importante es que el endpoint deja de retener esa sesión.

La relación completa es:

```
Petición HTTP
↓
get_db crea sesión
↓
endpoint usa sesión
↓
termina petición
↓
finally ejecuta db.close()
```

Resultado final del archivo

Después de cargar este archivo, queda disponible la función:

```
get_db()
```

Esta función puede ser utilizada por cualquier endpoint que necesite acceder a PostgreSQL.

Ejemplo:

```
def endpoint(db: Session = Depends(get_db)):
    ...
```

El comportamiento será:

1. FastAPI recibe una petición.
2. Detecta `Depends(get_db)`.
3. Ejecuta `get_db()`.
4. Se crea una sesión con `SessionLocal()`.
5. La sesión se entrega al endpoint mediante `yield`.
6. El endpoint usa la sesión.
7. Al terminar la petición, se ejecuta `finally`.
8. La sesión se cierra con `db.close()`.

7 Relación con el flujo técnico del laboratorio

`session.py` conecta los endpoints de FastAPI con la infraestructura de base de datos definida en `database.py`.

La relación técnica es:

```
database.py
↓
SessionLocal
↓
session.py
↓
get_db()
↓
Depends(get_db)
↓
endpoint FastAPI
↓
PostgreSQL
```

Dentro del flujo general del SIEM:

```
Evento entra por API
↓
Endpoint necesita base de datos
```

```
↓
FastAPI ejecuta get_db()
↓
SessionLocal crea sesión
↓
Endpoint guarda o consulta datos
↓
db.close() cierra la sesión al terminar
```

Este archivo es pequeño, pero muy importante porque evita gestionar sesiones de base de datos manualmente en cada ruta.

8 Errores típicos o puntos importantes

No cerrar la sesión

La función utiliza:

```
finally:
    db.close()
```

Esto garantiza que la sesión se cierre siempre.

Si no se cerraran las sesiones, con el tiempo podrían acumularse conexiones abiertas.

Confundir `yield` con `return`

En este archivo se usa:

```
yield db
```

No se usa:

```
return db
```

Esto es importante porque FastAPI permite usar dependencias con `yield` para ejecutar código de limpieza después de que termine la petición.

Si se usara `return`, sería más difícil garantizar el cierre automático de la sesión en este patrón.

`SessionLocal()` crea una sesión nueva por petición

Cada vez que un endpoint usa:

```
Depends(get_db)
```

se crea una nueva sesión para esa petición.

Esto evita compartir la misma sesión entre peticiones diferentes.

La sesión depende de `database.py`

Si `SessionLocal` está mal configurado en `database.py`, entonces `get_db()` también fallará.

Por tanto, errores en `DATABASE_URL`, `engine` o `sessionmaker` pueden afectar a todos los endpoints que usen `get_db`.

Anotación genérica `Generator`

La función está anotada como:

```
def get_db() -> Generator:
```


Esto indica que devuelve un generador, pero no especifica el tipo exacto que genera.

Una anotación más específica podría ser:

```
def get_db() -> Generator[Session, None, None]:
```

Pero para este proyecto, la versión actual es suficiente y más simple.

9 Comandos útiles relacionados

Probar que `get_db` puede importarse:

```
docker exec -it siem-api python -c "from app.db.session import get_db; print(get_db)"
```

Probar que `SessionLocal` puede crear una sesión:

```
docker exec -it siem-api python -c "from app.db.database import SessionLocal; db = SessionLocal(); print(db); db.close()"
```

Probar una consulta usando una sesión:

```
docker exec -it siem-api python -c "from app.db.database import SessionLocal; from sqlalchemy import text; db = SessionLocal(); print(db.execute(text('SELECT 1')).scalar_one()); db.close()"
```

Probar el endpoint `/health`, que usa `get_db`:

```
curl http://localhost:8000/health
```

Ver logs de la API:

```
docker logs siem-api
```

Ver logs en tiempo real:

```
docker logs -f siem-api
```

Reiniciar la API:

```
docker compose --env-file docker/.env -f docker/compose.yml restart api
```